2026 / SPRING

DRIVER ACTIVITY *RECOGNITION*

USING DEEP TEMPORAL MODELS

A. Batuhan Erdoğan
Advisor: Barış Çiçek

Department of Mathematics · Izmir Institute of Technology

Anguita et al. (2013); Zaki & Meira (2020)

Window-Based Activity Recognition

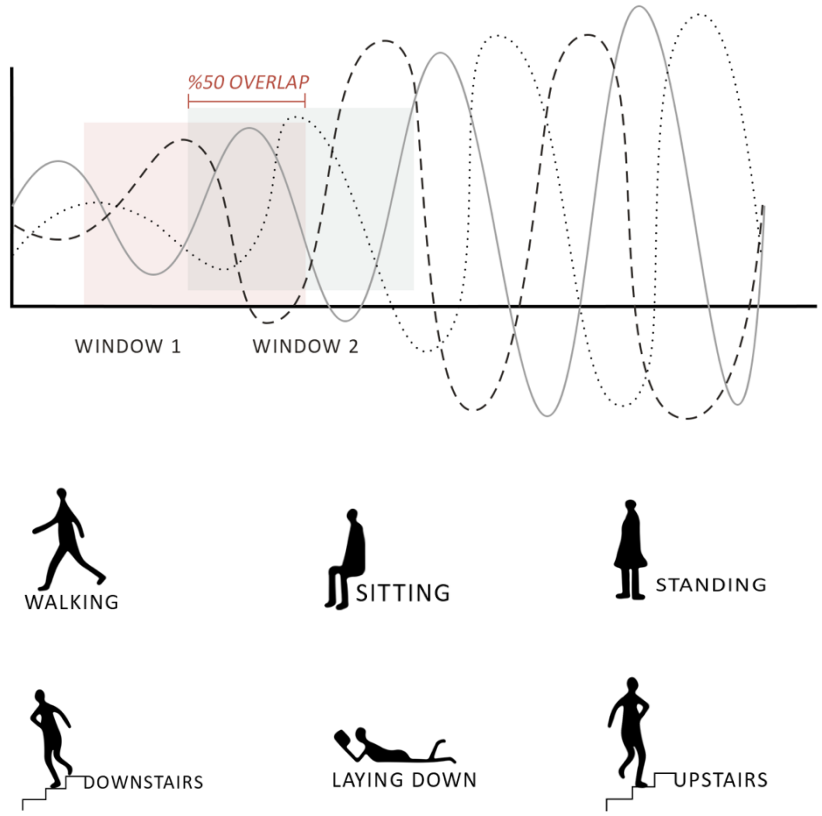
Recognizing 6 baseline human activities from smartphone sensor data.

THE UCI-HAR PROBLEM

Dataset: UCI Human Activity Recognition - 6 baseline activities (walking, sitting, standing, lying down, walking up/downstairs).
Data Source: Smartphone inertial sensors (accelerometer + gyroscope), captured at 30 Hz.
Samples: 10,299 labeled instances from 30 volunteers.

WINDOW-BASED APPROACH

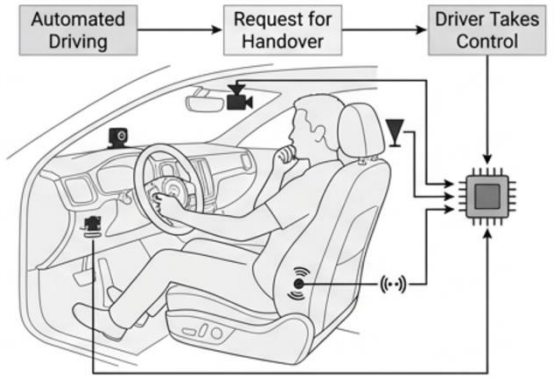
Raw sensor signal divided into **fixed-size sliding windows** (2.56 seconds each, 50% overlap).
From each window: **561 hand-crafted features** extracted (mean, std, frequency-domain coefficients, etc.).
Each window processed **INDEPENDENTLY**, no temporal connection to the previous or next window.
Model: Multinomial Logistic Regression (MLR).
Result: 98.25% test accuracy (surpassing the 96.2% literature baseline).



Martin et al. (2019), ICCV

Why Driver Activity Recognition?

Driver monitoring is a safety-critical task in conditionally automated vehicles.

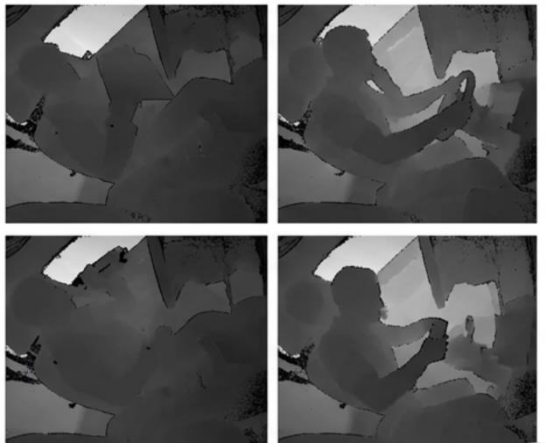


THE NEED

In conditionally automated vehicles (SAE Level 3), the human driver remains the fallback. The system must know **what** the driver is doing at all times. Not just "attentive vs. distracted" - we need fine-grained recognition of 34 distinct activities (eating, reading, using phone, entering/exiting car...). This is a safety-critical task: a driver reading a newspaper requires a different takeover protocol than one sitting idle.

WHO NEED THIS

Automotive industry: Advanced driver monitoring systems (DMS) for Level 3+ autonomy.
Insurance & fleet management: Driver behavior profiling for risk assessment.
Safety research: Understanding real in-cabin behavior for accident prevention.



EXISTING WORK

Drive&Act (Martin et al., 2019): The largest multi-modal driver behavior benchmark. Multiple approaches: 3D CNNs (C3D, P3D, I3D), body-pose methods, sensor-based approaches.
Challenge: Fine-grained activities are visually similar and context-dependent.

Martin et al. (2019), ICCV

From Window-Based to Temporal Models

We move from hand-crafted features to deep learning on raw video data.

WHY THE SHIFT?

The new problem: VIDEO data (raw pixel input). Activities unfold over time (e.g., reaching → pulling → pushing).
We can't hand-craft features from raw pixels → **need CNNs** for spatial extraction.
We can't ignore temporal order → **need LSTMs** for sequential modeling.

THE DRIVE&ACT DATASET

PROPERTY	VALUE
Subjects	15 (vp1 – vp15), 2 sessions each
Total recordings	30 full continuous videos
Activity classes	34 mid-level fine-grained classes
Total frames	~9.6 million
Frame rate	30 fps

MODALITIES (KINECT V2)

Infrared (IR): Captures texture and appearance (hand positions, object shapes, clothing).
Depth: Captures 3D geometry (body pose, spatial relationships, distances).

Both are **single-channel** (512×424), each pixel represents a single intensity or distance value.

Neural Networks: How Machines Learn

The artificial neuron, activation functions, and how networks process data.

ARTIFICIAL NEURON EQUATION

Computes a weighted sum, adds bias, applies activation.

$$(1)\quad \text{net}_k = b_k + \mathbf{w}_k^T \mathbf{x} \qquad (2)\quad z_k = f(\text{net}_k)$$

ACTIVATION FUNCTION

Introduce non-linearity.

ReLU: used in hidden layers: $(3)\quad f(x) = \max(0, x)$

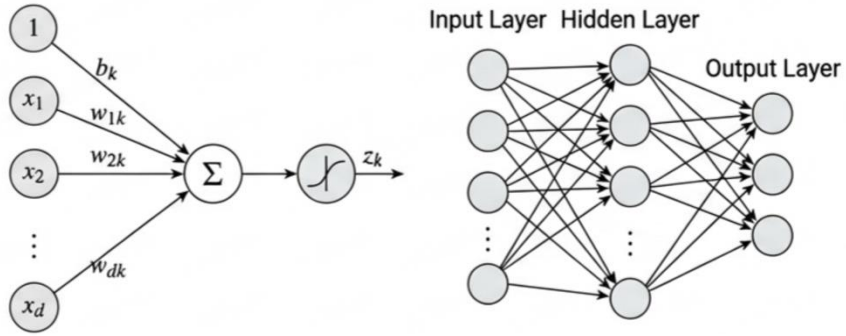
Sigmoid: outputs 0–1, used in LSTM Gates: $(4)\quad \sigma(x) = \dfrac{1}{1+e^{-x}}$

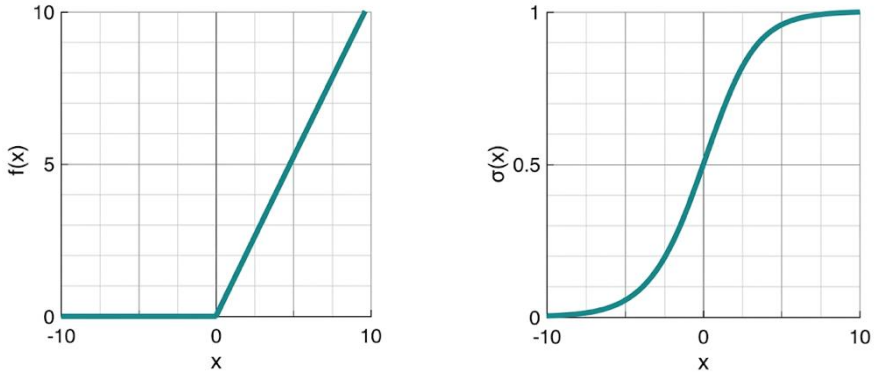
Softmax: output layer, converts scores to probabilities.

FEEDFORWARD NETWORK

Information flows forward without cycles:

$$(6)\quad \mathbf{o} = f\left(\mathbf{b}_o + W_o^T f\left(\mathbf{b}_h + W_h^T \mathbf{x}\right)\right)$$





ReLU & SIGMOID PLOTS

Loss Function and Training

How networks measure error and update weights.

LOSS FUNCTION

Cross-entropy measures prediction error. Penalizes low confidence in the correct class and rewards confidence in the correct class:
High prob. to correct class → low loss.
Low prob. → high loss (penalized).

$$\mathcal{E} = -\sum_{i=1}^{K} y_i \ln(o_i) \tag{7}$$

STOCHASTIC GRADIENT DESCENT

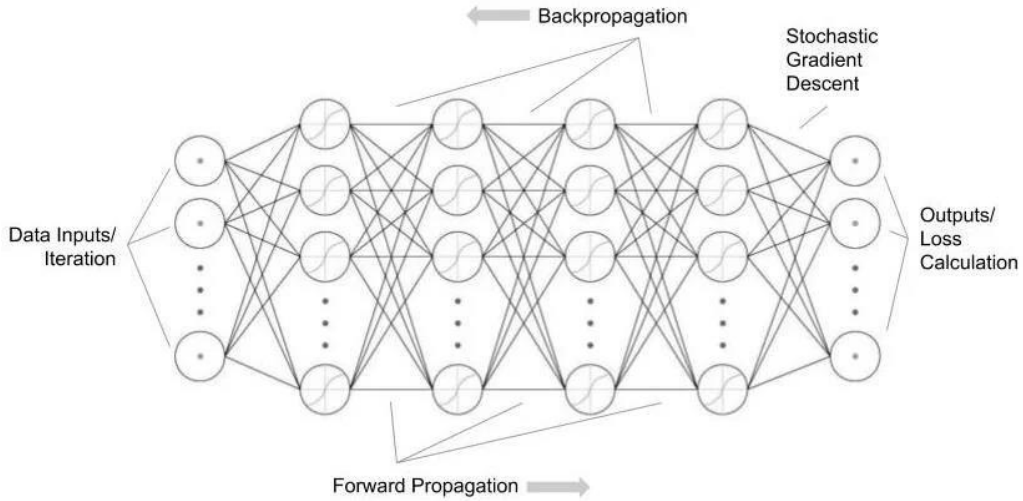
Adjust weights to reduce loss. In practice, each step estimates the gradient from a random mini-batch.
Noisy but fast, and the noise helps escape shallow minima. η is the learning rate (step size).

$$W \leftarrow W - \eta \nabla_W \mathcal{E} \tag{8}$$

BACKPROPAGATION

Uses the **chain rule** from calculus to propagate error gradients backward through the network layers.
Layer error δ^l is computed from the next layer's error.
One matrix multiply per layer, gradients reused.

$$\delta^l = \partial f^l \odot (W_l \, \delta^{l+1}) \tag{9}$$



How Computers See Images

From pixel matrices to convolutional networks. Why MLPs aren't enough?

IMAGES AS MATRICES

A grayscale image = a matrix of pixel intensities: $X \in \mathbb{R}^{H \times W}$ (1 channel).
RGB = three stacked matrices: $X \in \mathbb{R}^{H \times W \times 3}$

Our data: Kinect IR and Depth are each single-channel (512×424), each pixel is one number (intensity or distance). Resized to 224×224 for the CNN.

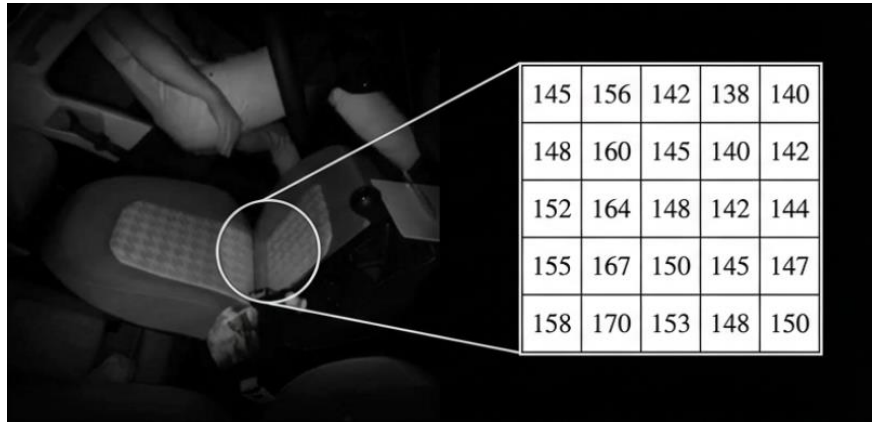
WHY STANDARD NETWORKS (MLP) FAIL

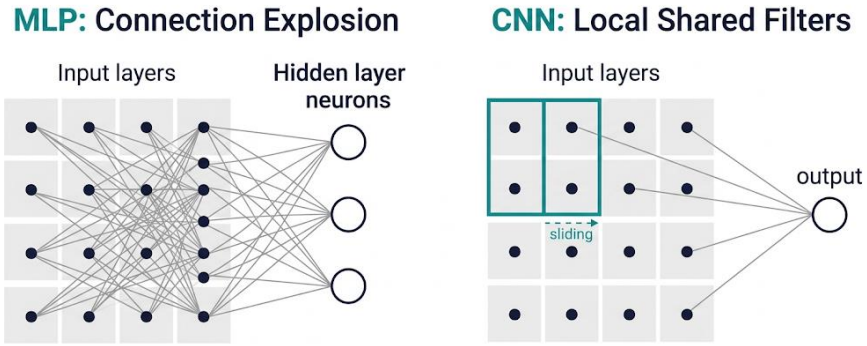
A 224×224×3 RGB image = **150,528 input values**. Fully connected to 1,000 hidden neurons = **150 million parameters** in layer 1.

Critically: no concept of spatial structure. It treats pixel (0,0) the same as pixel (112,112). MLP ignores that neighboring pixels are strongly related (edges, textures).

THE CNN IDEA

Instead of connecting every pixel to every neuron: Use a small **local filter**; share weights across all positions. The same edge detector works everywhere. Result: dramatically fewer parameters and built-in spatial pattern recognition.





Convolutional Neural Networks

How filters slide over inputs, with padding and stride controlling output size.

2D CONVOLUTION: A LOCAL DOT PRODUCT

A small **k×k filter** slides over the input, computing element-wise multiplication and summing at each position.

$$(10) \quad (X * W)_{i,j} = \sum_{a=1}^{k} \sum_{b=1}^{k} x_{i+a-1,\, j+b-1} \cdot w_{a,b}$$

THE FILTER

A learnable matrix of weights (e.g., 3×3 = 9 parameters).
Different filters detect different patterns: edges, corners, textures. **m filters** produce **m feature maps**.

$$(11) \quad Z^{l+1} = f\left((Z^{l} * W) + b\right)$$

PADDING

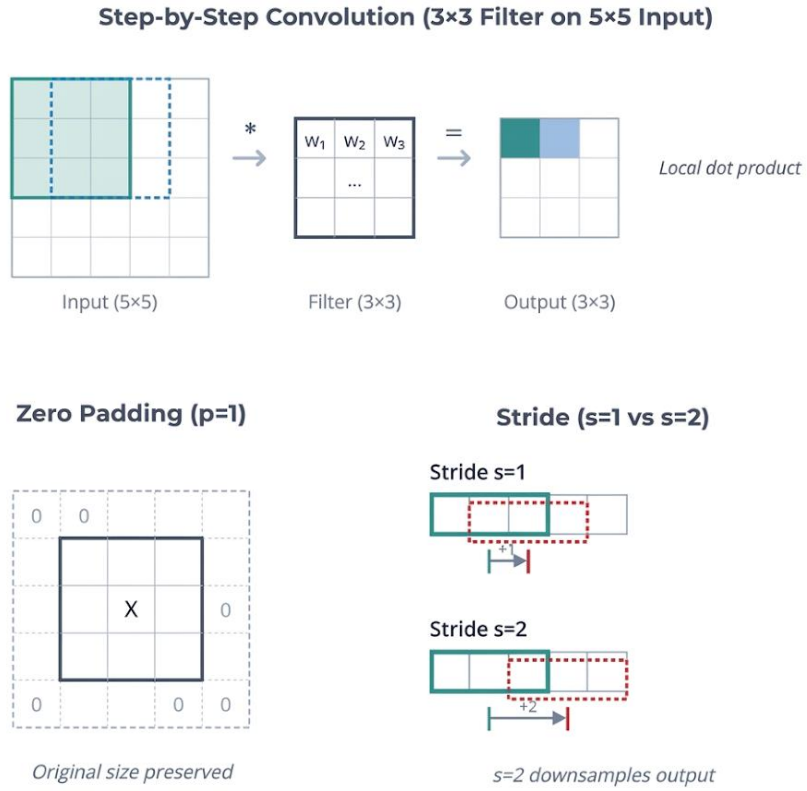
Without padding, border pixels are underrepresented. Same-padding adds zeros to preserve size.

$$(12) \quad p = \left\lfloor \frac{k-1}{2} \right\rfloor$$

STRIDE & OUTPUT

Stride (s): Step size. s=1: full resolution; s=2: halves dims (built-in downsampling).

$$(13) \quad n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$



Pooling and the CNN Pipeline

From spatial summarization to a hierarchy of learned features.

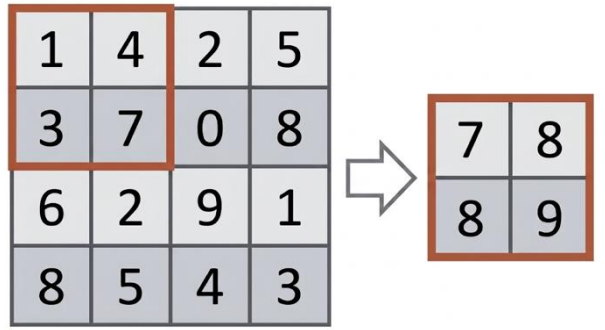
MAX POOLING: SPATIAL SUMMARIZATION

$$z_{i,j}^{\text{pool}} = \max_{a,b \in [1,k]} z_{(i-1)s+a,\,(j-1)s+b}^{l} \quad (14)$$

Within each k×k window, keep only the **maximum value** (typical: k=2, s=2).

Reduces spatial dimensions by half (224 → 112 → 56 → …).

Translation invariance: if a detected feature shifts slightly, the maximum value remains the same.



THE FULL CNN PIPELINE

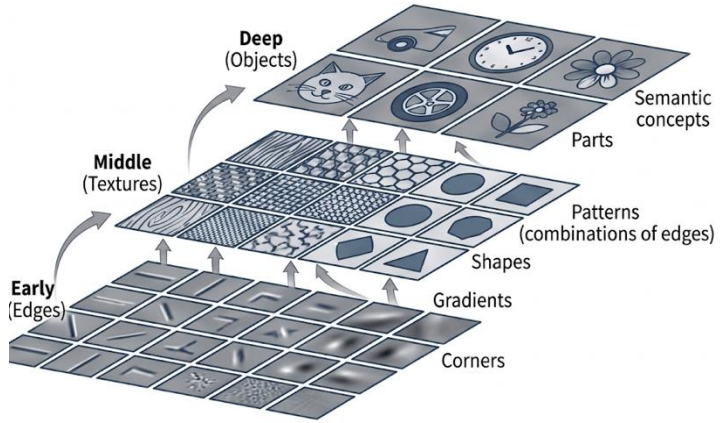
FEATURE HIERARCHY

Early layers: edges, corners, simple gradients.

Middle layers: shapes, textures, patterns.

Deep layers: objects, parts, semantic concepts.

This hierarchy emerges automatically from training.



He et al. (2016); Yosinski et al. (2014)

Deep Networks with Skip Connections

How residual connections enable training of very deep networks.

THE DEPTH PROBLEM

Deeper networks learn more complex features, but training is hard. **Vanishing gradients**: gradients vanish through many layers; early layers stop learning.

RESIDUAL (SKIP) CONNECTION

Instead of full transformation, learn the residual: what to ADD. If no change needed, F(x)=0, input passes unchanged. The identity matrix I ensures the gradient NEVER vanishes.

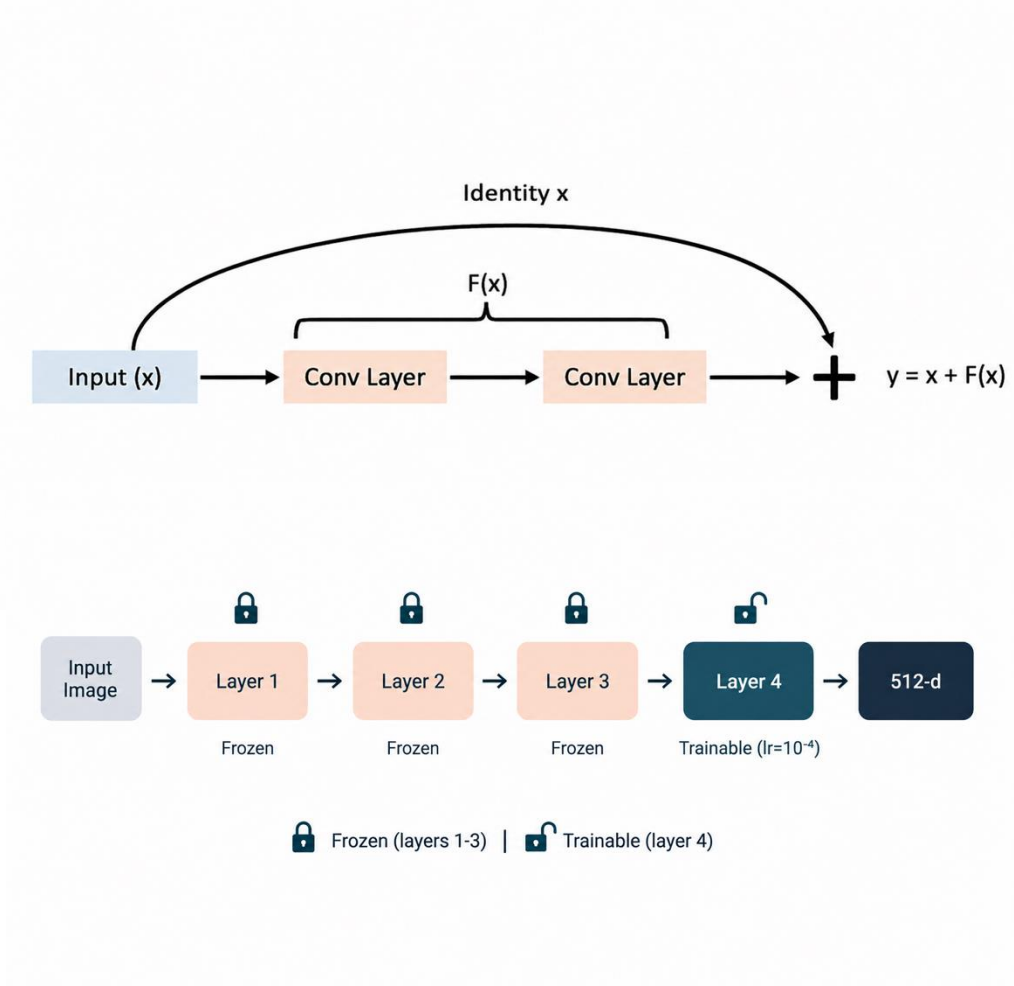
$$(15)\quad \mathbf{y} = F(\mathbf{x}) + \mathbf{x} \qquad (16)\quad \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + I$$

RESNET-18 BACKBONE

4 groups of residual blocks (layer1 → layer4); each with 2 blocks. Final output: **512-d feature vector**.

TRANSFER LEARNING

Pretrained on ImageNet: Freeze layers 1–3 · Fine-tune layer 4.



Hochreiter & Schmidhuber (1997); Bahdanau et al. (2015)

Long Short-Term Memory (LSTM)

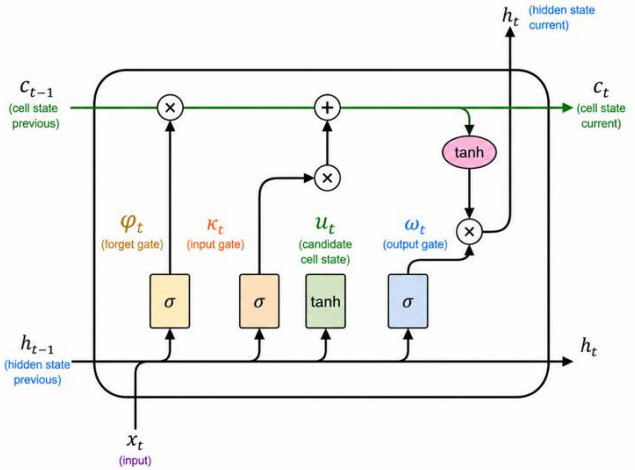
From spatial features to gated temporal modeling.

FROM SPATIAL FEATURES TO TEMPORAL MODELING

Activities unfold over time:

"entering car" = door opens → steps in → sits → closes door.

RNNs process sequential data via a hidden state, but suffer from vanishing gradients. LSTM solves this with a gated architecture and an additive memory update.



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

LSTM GATE EQUATIONS

Forget gate *keep old memory*

(17) $\varphi_t = \sigma\left(W_\varphi^T x_t + W_{h\varphi}^T h_{t-1} + b_\varphi\right)$

Input gate *write new info*

(18) $\kappa_t = \sigma\left(W_\kappa^T x_t + W_{h\kappa}^T h_{t-1} + b_\kappa\right)$

Candidate memory

(19) $c_t = \kappa_t \odot u_t + \varphi_t \odot c_{t-1}$

Cell update (additive) *prevents vanishing gradient*

(20) $u_t = \tanh\left(W_u^T x_t + W_{hu}^T h_{t-1} + b_u\right)$

Output gate

(21) $\omega_t = \sigma\left(W_\omega^T x_t + W_{h\omega}^T h_{t-1} + b_\omega\right)$

Hidden state

(22) $h_t = \omega_t \odot \tanh(c_t)$

The output gate decides what part of the cell state to expose as the hidden state h_t.

BILSTM & ATTENTION POOLING

BiLSTM: processes sequence forward and backward to capture past/future context.

(23) $h_t = \left[\overrightarrow{h_t}\ ;\ \overleftarrow{h_t}\right] \in \mathbb{R}^{512}$

Attention Pooling: learns which time steps matter most for the activity, replacing simple mean/max pooling with a weighted sum.

(22) $\mathbf{c} = \sum_{t=1}^{\tau} \alpha_t\, h_t$ (24) $\alpha_t = \text{softmax}(w_a^T h_t)$

Martin et al. (2019); He et al. (2016)

Proposed Two-Stage Pipeline

A spatial CNN + temporal LSTM architecture for fine-grained driver activity recognition.

STAGE 1: SPATIAL FEATURE EXTRACTION

Each video frame (IR and Depth) processed separately. Pretrained **ResNet-18** extracts a 512-dimensional feature vector per frame, per modality. Outputs concatenated into a single **1024-d vector** per time step.

STAGE 2: TEMPORAL MODELING

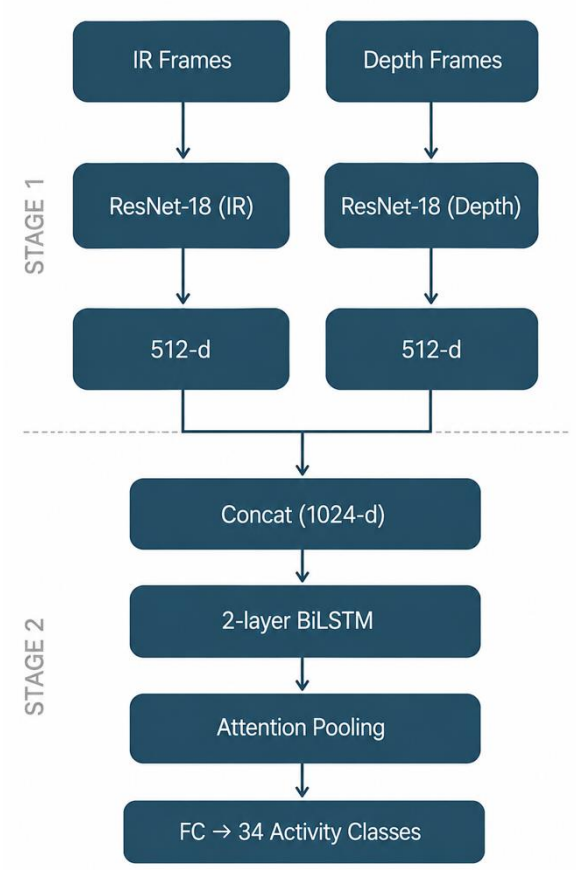
Sequence of 1024-d vectors fed into a **2-layer BiLSTM**. Attention pooling aggregates temporal information. Fully connected (FC) layer predicts **34 activity classes** via Softmax.

KEY DESIGN DECISIONS

Separating spatial & temporal: allows each component to be optimized independently.

Pretrained CNNs: prevents overfitting on limited dataset.

Chunk merging: consecutive same-activity chunks merged → context 3 s → 6 s, padding 53% → 6%.

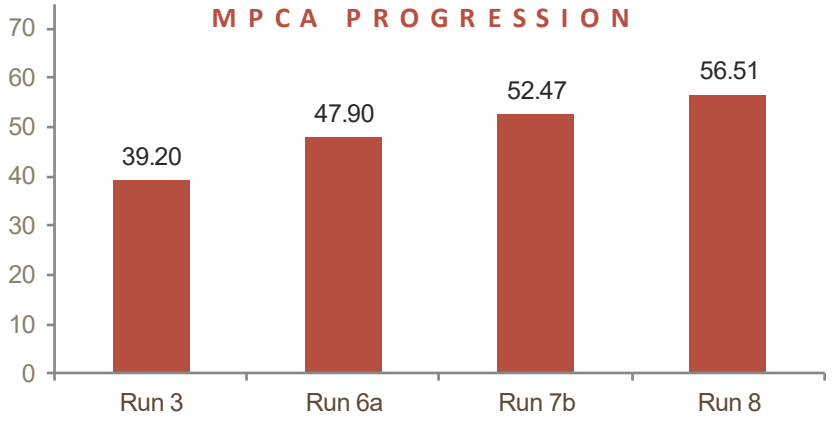


Martin et al. (2019), Table 5

Experimental Results

Ablation showing performance progression and three improvement axes (+17.31 pt total).

RUN	KEY CHANGE	MPCA	OVERALL	I3D GAP
Run 3	Frozen CNN baseline	39.20%	44.80%	29.31 pt
Run 6a	CNN fine-tuning (layer4)	47.90%	61.60%	20.61 pt
Run 7b (k=2)	Chunk merging (6s context)	52.47%	66.28%	16.04 pt
Run 8	**IR + Depth fusion**	**56.51%**	**63.98%**	**12.00 pt**



THREE IMPROVEMENT AXES · TOTAL: +17.31 PT

1 Spatial feature quality +8.70 pt

CNN fine-tuning

Largest single gain; adapts features from ImageNet to in-cabin domain.

2 Temporal context +4.57 pt

Chunk Merging

Simple preprocessing extending context to 6 seconds; zero added parameters.

3 Modality complementarity +4.04 pt

IR + Depth Fusion

Combining IR texture with Depth geometry significantly improves rare class recall.

Martin et al. (2019); *Carreira & Zisserman (2017); Tran et al. (2015); Qiu et al. (2017)*

Comparison with Published Methods

Our pipeline against state-of-the-art 3D CNN and body-pose methods on Drive&Act.

METHOD	TYPE	INPUT	MPCA
I3D (Carreira & Zisserman, 2017)	*3D CNN*	Kinect IR+Depth+Color	**68.51%**
I3D	*3D CNN*	Kinect IR	64.98%
I3D	*3D CNN*	Kinect Depth	60.52%
Ours (Run 8)	*2D CNN + LSTM*	Kinect IR+Depth	**56.51%**
Ours (Run 7b k=2)	*2D CNN + LSTM*	Kinect IR	52.47%
Three-Stream	*Body pose*	Kinect skeleton	46.95%
P3D (Qiu et al., 2017)	*3D CNN*	NIR front-top	45.32%
C3D (Tran et al., 2015)	*3D CNN*	NIR front-top	43.41%

WHAT ARE THESE METHODS?

C3D: 3D CNN applying 3×3×3 convolutions directly on video volumes to process 16-frame clips.

P3D: Pseudo-3D ResNet factorizing full 3D convolutions into separate spatial (2D) and temporal (1D) filters.

I3D: Inflated 3D ConvNet that expands 2D ImageNet-pretrained filters into 3D. State-of-the-art on Drive&Act.

KEY OBSERVATIONS

Our pipeline surpasses all body-pose methods AND 3D CNN baselines (C3D, P3D).

12 pt gap to I3D with only 4.2M params (vs. I3D's ~12M).

Caveats: C3D/P3D use NIR front-top sensor. Only I3D matches our Kinect modality. Literature uses 3-fold averages; ours uses split 0.

Martin et al. (2019), Table 5

Modality Ablation Study

Quantifying IR-only, Depth-only, and Fusion to confirm cross-modal complementarity.

MODALITY	MPCA	OVERALL
IR-only (Run 7b k=2)	52.47%	66.28%
Depth-only (Run 8a)	45.54%	57.85%
IR + Depth fusion (Run 8)	**56.51%**	**63.98%**

KEY FINDINGS & EXAMPLE CLASSES

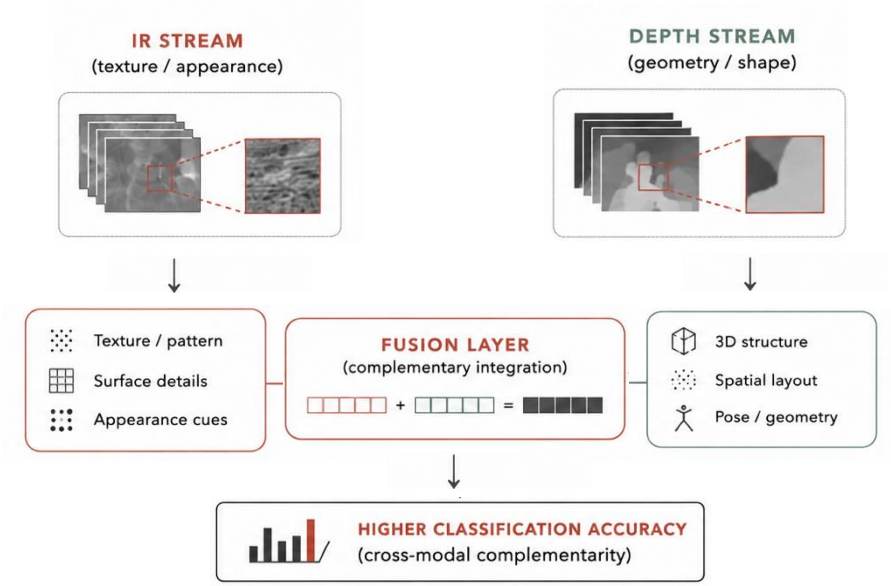
Fusion gain: Shows genuine cross-modal complementarity, not just added parameters.

IR excels at: Texture-dependent activities (hand-object interactions, phone, reading).

Depth excels at: Geometry-dependent activities (door operations, entry/exit, body pose).

EXAMPLE CLASS PERFORMANCE (IR + DEPTH = FUSION)

closing_bottle: 52% + 0% = **77%**
taking_off_sunglasses: 0% + 80% = **80%**
entering_car: 100% across all modalities



Challenges and Limitations

Three structural limitations of our 2D-CNN + LSTM approach.

TEMPORAL DIRECTION CONFUSION

Activities differing only in motion direction (opening vs. closing doors, putting on vs. taking off items) are confused.

2D CNN features capture per-frame appearance but **don't encode motion direction**.

A 3D CNN or optical flow would be needed to distinguish these pairs.

MPCA VS. OVERALL TRADEOFF

Fusion improves rare class recall but **eating drops from 56% to 26%** (confused with preparing_food due to similar depth geometry).

This is an inherent tension:

Optimizing for mean per-class accuracy redistributes model capacity toward rare classes, sometimes at the cost of frequent ones.

Standard regularization (dropout, weight decay, label smoothing) is applied but cannot fully bridge the domain gap between subjects

STRUCTURAL OVERFITTING

A **train-test gap** is observed due to strict subject-based evaluation. Test drivers are never seen during training, and individuals differ in posture, body shape, and movement style.

Why this split is necessary:

This split is both the benchmark standard and the only scientifically valid approach. A random split would leak driver identity into the test set, inflating accuracy without testing true generalization.

Martin et al. (2019); He et al. (2016); Hochreiter & Schmidhuber (1997)

A Lightweight Pipeline for Drive&Act

*A two-stage CNN + LSTM pipeline achieves **56.51% MPCA** on Drive&Act, narrowing the I3D gap to **12 percentage points** with a lightweight architecture.*

CONTRIBUTIONS

Two-stage IR+Depth pipeline:
Achieves 56.51% MPCA, surpasses all body-pose methods and 3D CNN baselines (C3D 43.41%, P3D 45.32%, Three-Stream 46.95%).

Chunk merging strategy:
Extending temporal context from 3 s to 6 s yielded +4.57 MPCA points with zero added parameters.

Systematic modality ablation:
Quantified IR-only (52.47%), Depth-only (45.54%), and Fusion (56.51%), confirming genuine cross-modal complementarity.

FUTURE DIRECTIONS

3-fold cross-validation:
Essential for robust evaluation and benchmarking against the full Drive&Act standard.

Transformer-based temporal encoder:
Replacing BiLSTM with self-attention mechanisms to better capture long-range dependencies and complex interactions.

Class-conditional adaptive fusion:
Implementing learned, per-activity modality weighting (depth for doors, IR for phones) rather than static concatenation.

References

1 **Martin, M. et al. (2019).** *Drive&Act: A Multi-Modal Benchmark for Fine-Grained Driver Behavior Recognition.* ICCV 2019.

2 **He, K. et al. (2016).** *Deep Residual Learning for Image Recognition.* CVPR 2016.

3 **Hochreiter, S. & Schmidhuber, J. (1997).** *Long Short-Term Memory.* Neural Computation, 9(8).

4 **Zaki, M.J. & Meira, W. (2020).** *Data Mining and Machine Learning.* Cambridge University Press.

5 **Carreira, J. & Zisserman, A. (2017).** *Quo Vadis, Action Recognition?* CVPR 2017.

6 **Bahdanau, D. et al. (2015).** *Neural Machine Translation by Jointly Learning to Align and Translate.* ICLR 2015.

7 **Yosinski, J. et al. (2014).** *How Transferable Are Features in Deep Neural Networks?* NeurIPS 2014.

8 **Tran, D. et al. (2015).** *Learning Spatiotemporal Features with 3D Convolutional Networks.* ICCV 2015.

9 **Qiu, Z. et al. (2017).** *Learning Spatio-Temporal Representation with Pseudo-3D Residual Networks.* ICCV 2017.

Thank you for your attention